



Advanced Terraform on Azure

Lab6a Module Foundations

Table of Contents

Expected Duration	2
Prerequisites.....	2
Part 1. Publishing & Consuming Terraform Modules	2
Teaching Points	2
Try It Yourself (Optional).....	2
Phase 0 – Review the lab environment	3
Phase 1 – Baseline: Deploy root module resources	3
Phase 2 – Reference the modules.....	4
Phase 3 – Introduce environments.....	6
Phase 4 - Sloppy code doesn't belong here!	7
Phase 5 – Publish modules as versioned artifacts	9
Phase 6 – Consume modules via “remote-style” sources.....	11
Phase 7 – Apply semantic versioning and upgrade safely	13
Part 2. Publishing & Consuming Git-Hosted Terraform Modules	15
Teaching Points	15
Phase 1 – Reconfigure the lab file structure	16
Phase 2 – Create and Publish the Module Repositories (GitHub).....	17
Phase 3 – Consume Git-hosted modules in DEV	19
Phase 4 – Replicate for Test and Prod	20
Phase 5 – Validate the End State.....	20
Part 3. Controlled Promotion of a Module Version	22
Phase 1 – Implement a Non-Breaking Network Module Change	22
Phase 2 – Promote Network Module v1.0.1	23

Expected Duration

Part 1 of this lab is expected to take between 45-55 minutes

Part 2 of this lab is expected to take 35-45 minutes

Part 3 of this lab is expected to take 20-25 minutes

Total expected lab time is therefore 100-125 minutes

Take breaks between parts as needed.

Prerequisites

This lab assumes the presence of a remote state backend, as created in Lab 4. If this does not exist in your lab Azure subscription, then run the following script in a VSC terminal session to create it ...

```
c:\qatfadva\artifacts\fixes\bootstrap-backend.ps1
```

Part 1. Publishing & Consuming Terraform Modules

Teaching Points

- Understand modules as standalone infrastructure products.
- Recognize the separation between module lifecycle and root lifecycle.
- Publish modules from Lab 5 into a shared location for reuse.
- Consume modules from a fresh root module with no monolith dependencies.
- Introduce remote module sources (local, git, registry).
- Apply semantic versioning to modules and manage upgrades safely.
- Use modules consistently across DEV/TEST/PROD with controlled version drift.

Try It Yourself (Optional)

Before following the guided steps, attempt to:

- Split your Lab 5 modules into a separate folder.
- Create a new empty root module that calls those modules.
- Convert the module source paths into remote-style paths.
- Implement a module versioning strategy.

Phase 0 – Review the lab environment

In this phase, the modules created in Lab 5 will be treated as standalone artifacts. A well-designed module should outlive the root module that created it. Publishing forces you to treat modules as products: versioned, documented, and reusable.

In platform teams, modules are treated like internal products: they evolve on their own lifecycle, are versioned and documented, and are consumed by many application teams. This phase is about shifting mindset from “a pile of .tf files” to “reusable, versioned infrastructure components.”

1. Review the **lab6** file restructuring ...

- There is a new ‘minimal’ root module at **lab6/guided**. This module defines the resource group to be used throughout this lab and is the only resource explicitly created at root. No modules are currently invoked.
- The networking and security modules created in lab 5 have been moved into the course **artifacts** folder, under **/local_modules**.
- The environment .tfvars files created in lab 5 have been also been moved into **artifacts**, under **/tfvars**.
- Sample **README.md** files have been included for all modules, including root. These will be discussed during the module publishing process.

Phase 1 – Baseline: Deploy root module resources

Preliminaries

This deployment establishes a minimal root: it creates only the resource group and nothing else. In many enterprise codebases, root contains little or no direct resource creation and instead acts as the composition layer that selects module versions, passes environment configuration, and wires module outputs into downstream modules.

2. Update **c:/qatfadvaz/lab6/guided/providers.tf** with your **subscription id** and with your remote storage account **suffix**.

Implementation

The current configuration has no environment specific named tfvars files. These will be re-introduced later. The remote backend configuration has been configured with a key ‘dev.tfstate’ and the env variable is defined as ‘dev’ in terraform.tfvars. So this deployment can be viewed as the ‘dev’ deployment. Future steps will re-introduce specific ‘test’ and ‘prod’ environment configurations.

7. Deploy the resource group to use as a baseline ‘root-only’ deployment...

az login

```
cd c:\qatfadvez\lab6\guided
terraform init
terraform plan
terraform apply --auto-approve
```

8. Confirm deployment of the resource group, tracked in **dev.tfstate** statefile.

You now have a working baseline plan against which you can compare the modularized configuration.

Phase 2 – Reference the modules

9. A new application team should consume modules cleanly. Root is the **system composer**: it wires modules together and owns cross-module dependencies. Modules remain isolated products with explicit inputs and outputs.
10. Invoke the modules using local relative paths to begin with. Update **lab6/guided/main.tf ..**

```
module "network" {
  source = "../../artifacts/local-modules/network"
}

module "security" {
  source = "../../artifacts/local-modules/security"
}
```

There are many indicated errors at this point.

11. Run **terraform init** followed by **terraform validate**.

The errors are expected: they represent the module contract being enforced. The modules need values for;

```
location
prefix
resource_group_name
base_tag
nsg_id
vnet_address_space
subnet_cidrs
```

Some values (Category A) are part of the root-to-module contract and the root **MUST** pass those values when calling the module....

```
location
prefix
```

resource_group_name
base_tags
any cross-module wiring.

In this case, the wiring is the id of the nsg, generated in the security module, which must be passed to the network module. Terraform modules are intentionally isolated. A module cannot consume another module's outputs directly. All integration happens in the root module, which acts as the system composer. This enforces explicit contracts, prevents hidden coupling, and enables safe reuse and versioning. This is the first time you see root acting as a composer: it consumes outputs from one module and passes them into another — modules never talk directly.

12. In **guided/main.tf**, update the module calls to supply needed values, drawn from **guided/terraform.tfvars** and **guided/locals.tf** ...

```
module "network" {
  source = "../../artifacts/local-modules/network"
  prefix = local.prefix
  location = var.location
  base_tags = local.base_tags
  resource_group_name = azurerm_resource_group.rg.name
  nsg_id = module.security.nsg_id
}

module "security" {
  source = "../../artifacts/local-modules/security"
  prefix = local.prefix
  location = var.location
  base_tags = local.base_tags
  resource_group_name = azurerm_resource_group.rg.name
}
```

13. Run **terraform validate** again to verify that the Category A parameter requirements are now satisfied.

This leaves the Category B parameters. Category B values are workload/environment configurations. In this lab we intentionally keep them outside the module so different environments can supply different CIDRs and NSG policies without changing module code

The minimum Category B configurations (environment-dependent) needed are ...

vnet_address_space
subnet_cidrs
nsg_rules

These parameters are defined by the module interface but supplied by the consuming root module. If the module does not define defaults, the root must pass values; otherwise Terraform will raise an error, as seen here.

Defaults are powerful but dangerous. If a module quietly creates minimal or placeholder resources, teams can accidentally deploy incomplete foundations. Use defaults intentionally and prefer validations or preconditions where empty or missing values would be unsafe.

Rather than updating terraform.tfvars, we will provide these necessary, environment specific parameters by (re-)introducing named-tfvars for different environments. Given the higher precedence of these files, they can override values in terraform.tfvars as well as providing additional values.

Phase 3 – Introduce environments

The current configuration has a single environment 'dev' with a single remote state file 'dev.tfstate'. We will now address this to regain the multi-env configuration we had by the end of lab 5. Four tfvars files have been pre-provided in the **artifacts/tfvars** folder, three of which are the same files used in lab 5, reflecting DEV, TEST and PROD environments. The fourth, bad-dev.tfvars is for testing.

14. Examine and then copy the four top-level .tfvars files from **artifacts/tfvars** into **lab6/guided**

The tfvars define env, vnet_address_space and subnet_cidrs parameter values. Env is already defined at root via terraform.tfvars but needs to be over-ridden, the other parameters are not, nor are they currently being passed into the modules.

15. Given that the named-tfvars files now supply ALL of the values needed when calling the modules, this renders **terraform.tfvars** redundant. Delete **terraform.tfvars**.
16. Tfvars files exist to supply values to populate variables. These variables must therefore be declared. Update **lab6/guided/variables.tf** with untyped declarations of the variables whose values, populated from the chosen named-tfvars file, are to be passed into the modules ...

```
variable "vnet_address_space" {}  
variable "subnet_cidrs" {}  
variable "allow_groups" {}  
variable "nsg_rules" {}
```

17. Update the **network** module invocation in **lab6/guided/main.tf** to pass the appropriate variables, populated from a specified named-tfvars file values, into the network module:

```
...
vnet_address_space = var.vnet_address_space
subnet_cidrs = var.subnet_cidrs
}
```

18. Update the **security** module invocation in **lab6/guided/main.tf** to pass the appropriate variables, populated from a specified named-tvars file values, into the security module:

```
...
allow_groups = var.allow_groups
nsg_rules = var.nsg_rules
}
```

19. Run **terraform validate** again; the configuration should now be valid
20. Deploy the three environments ..

```
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform plan --var-file=dev.tfvars
terraform apply --var-file=dev.tfvars --auto-approve

terraform init -reconfigure -backend-config="key=test.tfstate"
terraform plan --var-file=test.tfvars
terraform apply --var-file=test.tfvars --auto-approve

terraform init -reconfigure -backend-config="key=prod.tfstate"
terraform plan --var-file=prod.tfvars
terraform apply --var-file=prod.tfvars --auto-approve
```

You can read ahead whilst these deployments progress

Multi-environment is not just separate state — it is controlled drift. DEV may move faster, PROD moves slower. Later in this lab, versioned modules allow you to choose when PROD upgrades, without blocking DEV experimentation.

Phase 4 - Sloppy code does not belong here!

In the previous phase, the root module declares some variables using empty blocks ({}).

This works, but it is *sloppy*. An empty variable declaration, known as an **untyped variable**, tells Terraform: “Accept any value, of any shape, and don’t validate it here.” When environment configuration is supplied via tfvars, this means the root module no longer enforces the shape or intent of that configuration. Incorrect values can pass through the root unchecked and only fail later, deep inside a child module, often with confusing or indirect error messages.

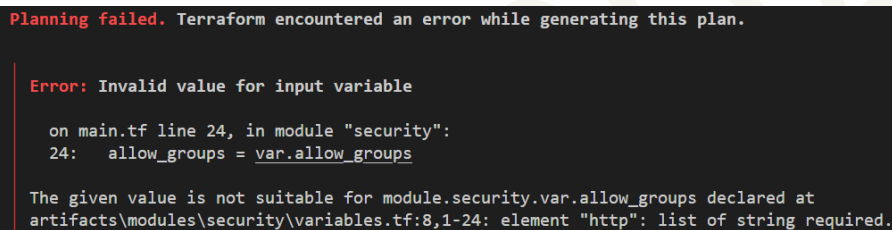
In real Terraform codebases, the root module represents the policy and composition boundary. Environment-owned configuration should be modelled explicitly and validated as early as possible. Using untyped {} variables hides mistakes instead of preventing them.

In this phase we will prove this sloppiness and then replace these placeholder declarations with strongly typed variables. The behaviour of the infrastructure does not change — but Terraform will now fail fast, with clear, actionable errors when environment configuration is incorrect.

In the previous phase we introduced **four** tfvars files. Open and review **bad_dev.tfvars**. Try to access what is 'bad' about this configuration. Its issues will surface as soon as we try to apply it.

21. Once the previous deployments are complete, apply the bad tfvars file;

```
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform plan --var-file=bad_dev.tfvars
```



```
Planning failed. Terraform encountered an error while generating this plan.

Error: Invalid value for input variable

   on main.tf line 24, in module "security":
   24:   allow_groups = var.allow_groups

The given value is not suitable for module.security.var.allow_groups declared at
artifacts\modules\security\variables.tf:8,1-24: element "http": list of string required.
```

The issue is in the message in bad_dev.tfvars, allow_groups is declared as a **map(string)** instead of **map(list(string))** as is defined in the security module.

With root variable allow_groups declared as {} (untyped), Terraform will happily load this and the failure occurs later when the module tries to iterate/validate.

22. Redefine the allow_groups variables at root ...

```
variable "allow_groups" {
  description = "Named CIDR allow-lists that NSG rules can reference."
  type       = map(list(string))
  default    = {}
}
```

23. Plan again and verify that the error is now caught at the root boundary (during variable type checking), rather than later inside the module...

```

Error: Invalid value for input variable

on bad_dev.tfvars line 5:
5: allow_groups = {
6:   http       = "10.0.0.0/24"
7:   https      = "10.0.2.0/24"
8:   admin      = "172.16.0.10/32"
9:   breakglass  = "192.168.99.99/32"
10: }

The given value is not suitable for var.allow_groups declared at variables.tf:21,1-24: element "http": list of string required.

```

24. All root variables used for module tfvars invocations should be strongly typed. Redefine the remaining untyped variables ...

```

variable "nsg_rules" {
  description = "Map of NSG rules to create."
  type = map(object({
    priority    = number
    direction   = string
    access      = string
    protocol    = string
    destination_ports = list(string)
    source_cidrs = list(string)
    allow_groups = optional(list(string), [])
  }))
}

variable "vnet_address_space" {
  type    = string
  description = "CIDR for the Virtual Network address space."
}

variable "subnet_cidrs" {
  type    = map(string)
  description = "Map of logical subnet name to CIDR."
}

```

Phase 5 – Publish modules as versioned artifacts

In this phase we stop treating shared-modules as simple folders and start treating them as versioned products. In enterprise Terraform, modules are deliverables: published, versioned, documented, and consumed by teams who never touch the module source.

24. Sample README.md files have been provided in each module for documenting purpose, required inputs, optional inputs, outputs, and known limitations.

Why ? A module without documentation is not a product. Publishing forces explicit contracts and safe reuse. README files are a fundamental part of professional Terraform projects because they document **intent, responsibility, and safe usage** in a way that code alone cannot. A well-written README explains what a configuration is responsible for, what it deliberately does not manage, how modules are composed, how environments differ, and how change is controlled through versioning and state

isolation. By making these decisions explicit, READMEs support predictable behaviour, safer upgrades, and consistent use across environments, which is essential for scalable, enterprise-grade Infrastructure as Code.

Implementation

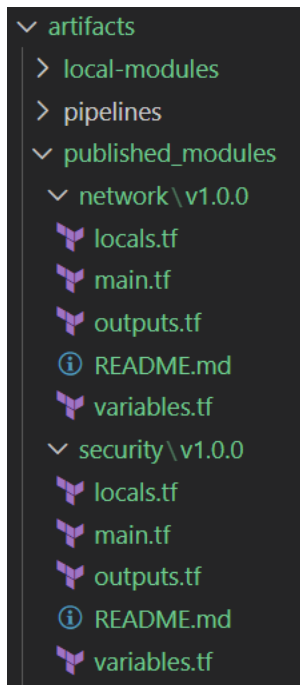
24. Create a **published-modules** folder in the course-wide **artifacts**

25. Create versioned paths:

published-modules/network/v1.0.0

published-modules/security/v1.0.0

26. Copy the **artifacts/local-modules** content into the appropriate version v1.0.0 folders, resulting in the file structure ...



To generate version differences moving forward, we need to identify a small, safe change we can make to the module contract. Let us add a module-specific manager label that can be updates as we go from v1.0.0 to v1.0.1 etc. This is simulating more detailed changes that might occur as we go through module revisions.

27. In **published-modules/security/1.0.0/locals.tf**, add a module tag that will merge with the base-tags;

```
mod_tags = merge(
  var.base_tags,
  {
    manager = "Michael Coulling-Green (SecMod v1.0.0)"
  }
)
```

)

Note. Adding the version details is purely cosmetic, allowing clarity as we progress through lab changes.

28. Repeat the above step in **published-modules/network/1.0.0/locals.tf** ...

```
manager = "Michael Coulling-Green (NetMod v1.0.0)"
```

To surface the tags being used by modules across different versions, we can generate module outputs to root ...

29. In **published-modules/network/v1.0.0/outputs.tf**, add a new output (non-breaking) to surface the tags in use ...

```
output "module-tags" {  
  value   = local.mod_tags  
  description = "Module tags (teaching aid)."  
}
```

31. Repeat the above step in **published-modules/security/v1.0.0/outputs.tf**

32. Surface the version outputs in root by creating **lab6/guided/outputs.tf** with the following content ..

```
output "network-module-tags" {  
  value = module.network.module-tags  
  description = "Tags used in Network module"  
}  
  
output "security-module-tags" {  
  value = module.security.module-tags  
  description = "Tags used in Security module"  
}
```

Takeaway

Modules are now treated as versioned artifacts rather than refactoring by-products.

Phase 6 – Consume modules via “remote-style” sources

We will now update root to consume modules via versioned paths, not direct shared folders, “simulating remote consumption” without requiring Git or Terraform Registry. The point is the pattern: **pin to a version**.

33. Open **lab6/guided/main.tf**.

34. Locate the two module blocks. They currently reference:

```
source = "../../../artifacts/local-modules/network"
source = "../../../artifacts/local-modules/security"
```

35. Update both module sources to point at the published, versioned folders:

```
module "network" {
  source = "../../../artifacts/published-modules/network/v1.0.0"
  ...
}

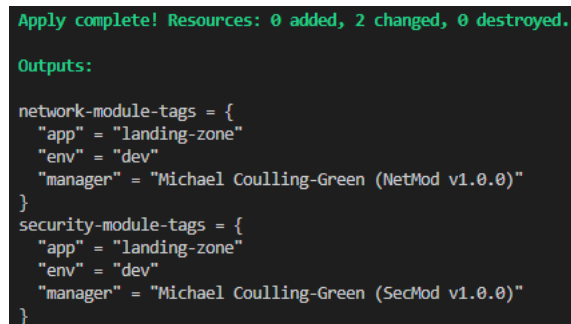
module "security" {
  source = "../../../artifacts/published-modules/security/v1.0.0"
  ...
}
```

36. Deploy v1.0.0 to DEV, TEST, and PROD, expecting two changes, the addition of the new tag to the Vnet and the NSG ...

```
terraform init -reconfigure -backend-config="key=dev.tfstate"
```

Note the updating of modules/modules.json in ./terraform

```
terraform apply --var-file=dev.tfvars --auto-approve
```



```
Apply complete! Resources: 0 added, 2 changed, 0 destroyed.

Outputs:

network-module-tags = {
  "app" = "landing-zone"
  "env" = "dev"
  "manager" = "Michael Coulling-Green (NetMod v1.0.0)"
}
security-module-tags = {
  "app" = "landing-zone"
  "env" = "dev"
  "manager" = "Michael Coulling-Green (SecMod v1.0.0)"
}
```

```
terraform init -reconfigure -backend-config="key=test.tfstate"
```

```
terraform apply --var-file=test.tfvars --auto-approve
```

```
terraform init -reconfigure -backend-config="key=prod.tfstate"
```

```
terraform apply --var-file=prod.tfvars --auto-approve
```

37. Confirm the deployments succeed with two updates and the module source paths now reference published-modules/.../v1.0.0

You have moved from “modules as local folders” to “modules as versioned artifacts.” This is the same mindset used when consuming them from Git or Terraform Registry: **the root module pins a module version.**

Phase 7 – Apply semantic versioning and upgrade safely

We will now demonstrate safe module upgrades with controlled version drift. We will create a module change, publish a new version, and upgrade DEV first—without forcing TEST or PROD to follow immediately.

This phase demonstrates controlled version drift: DEV upgrades first, while TEST and PROD remain pinned until an explicit upgrade is performed later

We will implement a small, safe v1.0.1 change to the security module contract, changing the manager tag value.

38. Create a new version folder for the security module:

artifacts/published-modules/security/v1.0.1

39. Copy security v1.0.0/* into v1.0.1/.

40. In **published-modules/security/1.0.1/locals.tf**, update the manager tag ...

```
mod_tags = merge(
  var.base_tags,
  {
    manager = "John Robinson (SecMod v1.0.1)"
  }
)
```

41. Upgrade **DEV** by updating the **security** module version in **guided/main.tf** from **v1.0.0** to **v1.0.1** and then running ...

terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform plan --var-file=dev.tfvars

```
Changes to Outputs:
~ security-module-tags = {
  ~ manager = "Michael Coulling-Green (v1.0.0)" -> "John Robinson (v1.0.1)"
    # (2 unchanged attributes hidden)
}
```

terraform apply --var-file=dev.tfvars --auto-approve

```

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:

network-module-tags = {
  "app" = "landing-zone"
  "env" = "dev"
  "manager" = "Michael Coulling-Green (NetMod v1.0.0)"
}
security-module-tags = {
  "app" = "landing-zone"
  "env" = "dev"
  "manager" = "John Robinson (SecMod v1.0.1)"
}

```

42. The output should confirm DEV consumes the newer security module in that the manager details have been updated. while test and prod remain unchanged. Use the Portal to verify the DEV NSG now has a module-level tag alongside the base tags...

Tags ([edit](#)) : app : landing-zone env : dev manager : John Robinson (SecMod v1.0.1)

43. Downgrade DEV by updating the **security** module version in **guided/main.tf** from **v1.0.1** to **v1.0.0** and then running ...

```

terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform apply --var-file=dev.tfvars --auto-approve

```

Takeaway

Semantic versioning enables safe, incremental upgrades without forced global change. In this phase, module version changes were made by manually updating the module source in main.tf. This is intentional. Terraform resolves module sources during terraform init, which means module sources must be statically defined in configuration and cannot be selected dynamically via tfvars. Keeping module versions explicit in code makes structural changes visible, reviewable, and auditable, rather than hidden behind runtime variables.

In real-world environments, this manual step is not performed directly on production code. Instead, module version changes are introduced through Git workflows: developers update the pinned version in a branch, validate the change in DEV, and promote it through TEST and PROD via pull requests and approvals. Part 2 introduces Git-based module sources, where this same version pinning becomes part of a controlled promotion pipeline rather than a manual edit.

Part 2. Publishing & Consuming Git-Hosted Terraform Modules

Teaching Points

In Part 1 you modularised a Terraform configuration and treated modules as standalone, reusable artifacts. Now you take the next step: modules are no longer consumed locally. Instead, they are published to Git and consumed remotely, while multiple environments exist simultaneously. We will demonstrate how real platform teams distribute Terraform modules and consume them safely across environments *without* CI/CD pipelines or automation.

By the end of this part of the lab, you will understand how to:

- Treat Terraform modules as versioned products hosted in Git
- Consume modules from a remote Git repository using pinned versions
- Operate **DEV, TEST, and PROD simultaneously** using separate state files
- Keep naming and tagging contracts stable while changing distribution mechanisms
- Apply **Pattern A** for environment separation (parallel roots, parallel state)

We intentionally avoid automation here; promotion is manual so that module lifecycle concepts remain visible.

Pattern A – Parallel Environment Roots

In this course, **Pattern A** refers to an environment separation strategy where:

- Each environment (DEV, TEST, PROD) has its own Terraform root directory
- Each environment maintains its own independent remote state file
- All environments consume Terraform modules using explicitly pinned versions
- Promotion between environments is a conscious action, not an automatic side-effect

This pattern allows environments to exist **simultaneously**, with clear ownership of state and no hidden coupling. It is commonly used by platform teams where multiple environments must remain live at all times.

In this lab, Pattern A is implemented using:

- Separate root folders (envs/dev, envs/test, envs/prod)
- Separate backend state keys (dev.tfstate, test.tfstate, prod.tfstate)

Pattern A in Context

There are several common ways teams separate Terraform environments.

Pattern A – Parallel Roots, Parallel State

Each environment has its own root configuration and its own state file. Environments exist simultaneously and are operated independently.

Pattern B – Single Root, Environment Switching

A single root configuration is reused with different variable files or workspaces to represent environments. Typically, only one environment is operated on at a time.

Pattern C – Branch-Based Environments

Each environment exists in a separate Git branch. This couples environment lifecycle to version control workflows and is generally discouraged for Terraform.

Why ? Branch-based environments are discouraged for Terraform because they couple long-lived stateful infrastructure to short-lived version-control workflows, increasing the risk of accidental promotion, state corruption, and unsafe rollback.

This lab uses **Pattern A** because it most closely reflects how platform teams operate long-lived DEV, TEST, and PROD environments in parallel, with explicit ownership of state and minimal hidden coupling.

Why platform teams prefer Pattern A

Pattern A avoids hidden coupling between environments. Each environment has explicit ownership of its own state, lifecycle, and failure domain. A broken DEV deployment cannot corrupt PROD state, and no environment is implicitly “current” based on workspace or branch context.

This part of the lab is organised into the following phases:

Phase 1 - Reconfigure the lab file structure

Phase 2 - Consume Git-hosted modules in DEV

Phase 3 - Replicate for TEST and PROD

Phase 4 - Validate the end state

Phase 1 – Reconfigure the lab file structure

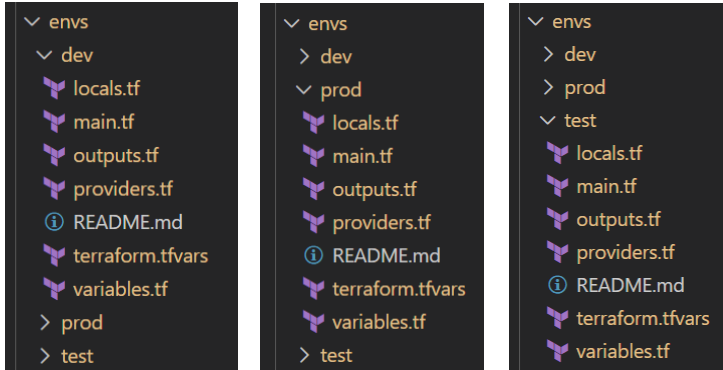
We will first restructure our files in order to adopt **Pattern A Environment Separation** as discussed above. Following the next steps carefully as misconfiguration here will cause major knock-on effects.

1. At **lab6/guided**, create the following folder structure ..

```
envs/dev
envs/test
envs/prod
```

2. Delete file **bad-dev.tfvars** as it is no longer needed.

3. Copy all **.tf**, **.tfvars** and **.md** files from **lab6/guided** into **each** envs **subfolder**.
4. Retaining the appropriate {env}-tfvars file in each subfolder, delete the two others.
5. **Rename** the remaining tfvars to **terraform.tfvars** ...



6. **Sanity Check** - Verify that each terraform.tfvars declares the correct environment; dev, test or prod.
7. Delete **all** files and folders, except **/envs** from **lab6/guided**.

Phase 2 – Create and Publish the Module Repositories (GitHub)

In this phase, you publish the modules created earlier to a GitHub repository. From this point onward, **Git becomes the distribution mechanism for modules**. Environments will no longer reference modules via local filesystem paths.

No infrastructure is deployed in this phase.

You will publish two separate repositories:

- terraform-azure-module-network
- terraform-azure-module-security

Important rules (apply to both repositories):

- The module repository must contain only module code (no root configuration).
- Do not include provider blocks in the module.
- Do not include terraform { backend ... } blocks in the module.
- Tag the first release as v1.0.0

8. Download a Personal Access Token from your Github account.
9. Create the GitHub repository for the network module ...

In GitHub, create a new repository named: **terraform-azure-module-network**

Create the repository empty (do not add a README, .gitignore, or license).

10. Clone the repository locally (c:\module-repos is used throughout) ...

```
cd c:\
mkdir module-repos
cd module-repos
git clone {your network repo url}
cd terraform-azure-module-network
```

11. Add the repo folder to VSC Explorer; File > Add Folder to Workspace > c:\module-repos

12. Copy the **artifacts/published-modules/network/v1.0.0** files into your cloned repository ...

```
terraform-azure-module-network/
main.tf
locals.tf
variables.tf
outputs.tf
README.md
```

13. Create a repository **.gitignore** file with the following contents:

```
.terraform/
*.tfstate
*.tfstate.*
crash.log
```

This prevents accidental commits of local Terraform artifacts.

14. Set Github global settings (once only configuration change)

```
git config --global credential.helper manager
git config --global credential.useHttpPath true
```

15. Commit, tag, and push v1.0.0 of the network module...

```
cd c:\module-repos\terraform-azure-module-network
git config --local user.email "{your repo user email}"
git config --local user.name "{your repo username}"
git remote set-url origin https://{repo username}@github.com/{repo
username}/terraform-azure-module-network.git

git status
git add .
git commit -m "Publish module v1.0.0"
```

```
git tag v1.0.0
git push origin main
```

Supply your PAT when prompted

```
git push origin v1.0.0
```

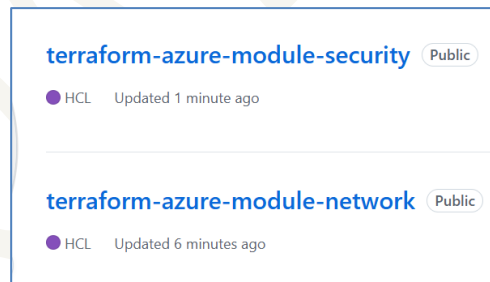
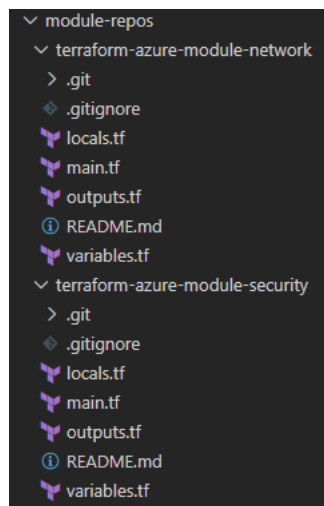
16. In GitHub, confirm:

- The repository contains the module files at the repository root.
- The v1.0.0 tag exists under Tags/Releases.

17. Repeat steps 9-15 (skipping step 14) for the security module, using **terraform-azure-module-security** as the repository name, again cloning locally under **c:\module-repos** and using v.1.0.0 artifacts

At the end of this phase:

- Two independent module repositories exist in GitHub.
- Both repositories are tagged at v1.0.0.
- No infrastructure has been deployed yet.



Phase 3 – Consume Git-hosted modules in DEV

In this phase, you reconfigure the **DEV** environment to use Git-hosted modules. Dev should always be the first consumer of new/moved modules and versions. It is where contracts are validated before TEST or PROD are touched.

18. In **lab6/guided/env/dev/main.tf**, update the two module source references to match your repo name.

network module:

"git::https://github.com/{repo}/terraform-azure-module-network.git?ref=v1.0.0"

security module:

"git::https://github.com/{repo}/terraform-azure-module-security.git?ref=v1.0.0"

19. In **lab6/guided/envs/dev/providers.tf**, verify the backend key is **dev.tfstate**
20. At **lab6/guided/envs/dev**, run **terraform init**
21. Note that we no longer need to specify the backend config during initialization.
22. Note that the remote modules are pulled down from github to .terraform\modules
23. Create a plan **terraform plan**
24. This should be a no-op plan, given that all resources are currently deployed

Phase 4 – Replicate for Test and Prod

In this phase, you repeat the above process for the TEST and PROD environments.

Before doing so, you **must** update the backend key specified in each environment **provider.tf** file. It is currently **dev.tfstate** and must be changed to **test.tfstate** or **prod.tfstate**

25. Update the two **provider.tf** files appropriately and then repeat Phase 3, adjusting as necessary for TEST and then PROD. (Hint: main.tf is identical across the three environments)

All planning should result in a no-op.

Phase 5 – Validate the End State

In this phase, you verify that environment separation, module reuse, and state isolation have been implemented correctly. This is not a functional test of Azure resources; it is a **platform correctness check** before moving forward.

26. From the **DEV** environment folder, run **terraform state list**
27. Confirm that only DEV resources appear. No TEST or PROD resource names are present
28. Repeat the same command from **envs/test** and **envs/prod**. Each environment must report **only its own resources**.

State isolation is the foundation of safe multi-environment Terraform. If environments can “see” each other’s resources, promotion and rollback become unsafe.

29. In the Azure storage account, inspect the state container.

Confirm that **four distinct state blobs exist**:

default.tfstate

dev.tfstate

test.tfstate

prod.tfstate

30. Delete **default.tfstate** as it is now redundant

31. Confirm that:

Each environment updates only its own state file

No environment overwrote or merged state with another

Terraform environments are separated by state, not by folders or branches. The folder structure is human-friendly; the backend is authoritative.

32. In the Azure portal, inspect resources across all three environments.

33. Confirm that:

- Resource names differ only by environment prefix
- Tags are consistent across DEV, TEST, and PROD
- The network and security modules added a manager tag to the Vnet and NSG
- No tags were manually passed into modules

Stable contracts allow module distribution mechanisms to change without affecting consumers. This is what enables safe reuse at scale.

By the end of this phase:

- DEV, TEST, and PROD exist simultaneously
- Each environment has an independent state file
- All environments consume explicitly pinned module versions
- Naming and tagging are derived, not duplicated
- No environment has been “promoted” automatically
- Each environment was deployed **intentionally and independently**.

Key Takeaways

- Git replaces the filesystem as the module distribution mechanism
- Root modules remain the system of composition
- Environment separation is achieved through state, not branching
- Stable contracts allow distribution mechanisms to change safely

Part 3. Controlled Promotion of a Module Version

You have published Terraform modules to Git and consumed them remotely across multiple environments. Now you evolve a single module and promote that change across DEV, TEST, and PROD in a controlled and intentional way.

The focus is promotion discipline and version control. No automation is introduced.

This part of the lab is organised into the following phases:

Phase 1 – Implement a Non-Breaking Network Module Change (v1.0.1)

Phase 2 – Promote Network v1.0.1 to DEV

Phase 3 – Approve Promotion

Phase 4 – Promote to TEST and PROD

Phase 5 – Validate the End State

Phase 1 – Implement a Non-Breaking Network Module Change

In this phase, you simulate a platform team releasing a non-breaking enhancement to a shared module and publishing it as a new version.

What “non-breaking” means here

You must **not** change the module interface in a way that would force consumers to edit their root modules.

Safe examples:

- Add a new output
- Add a new optional variable (with a default)
- Add an additional tag (derived internally)

Avoid:

- Renaming/removing variables
- Changing variable types
- Removing outputs
- Making an optional variable required

The new module version introduces a non-breaking enhancement by updating a manually generated tag that identifies the module manager. No consuming environment needs to change its inputs.

1. Open **module-repos\terraform-azure-module-network\locals.tf**

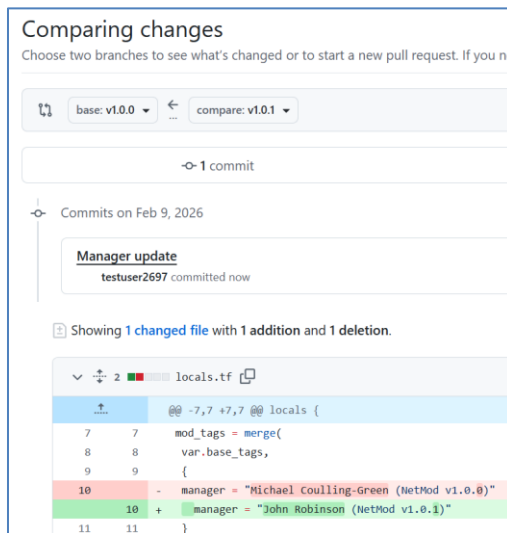
2. Update the manager tag assigned to the module. This is the non-breaking change in this simplified demonstration..

```
manager = "John Robinson (NetMod v1.0.1)"
```

3. Push the change to github and tag it ...

```
cd c:\module-repos\terraform-azure-module-network
git status
git add locals.tf
git commit -m "Manager update"
git tag v1.0.1
git push origin main
git push origin v1.0.1
```

4. Verify in GitHub that the v1.0.1 tag exists for the network module.



Phase 2 – Promote Network Module v1.0.1

In this phase you promote the new network module v1.0.1 through DEV/TEST/PROD. The security module remains pinned at v1.0.0.

5. In `lab6\guided\envs\dev\main.tf`, update the **network** module source reference:

Change `?ref=v1.0.0` to **`?ref=v1.0.1`**

6. At `lab6\guided\envs\dev`, re-initialise Terraform to fetch the new module version:

```
cd c:\qatfadavaz\lab6\guided\envs\dev
terraform init -upgrade
```

Review `.terraform/modules/modules.json` to verify the version update.

Review **.terraform/modules/network/locals** to verify it contains the updated manager tag

7. Create a plan using the DEV variable file:

terraform plan

Inspect the plan output and confirm that only network-managed resources show tag changes.

You should see changes similar to:

```
Changes to Outputs:
~ network-module-tags = {
  ~ manager = "Michael Coulling-Green (v1.0.0)" -> "John Robinson (v1.0.1)"
    # (2 unchanged attributes hidden)
}
```

8. Apply the change:
terraform apply --auto-approve
9. Repeating steps 5-8, with appropriate name adjustments, promote v1.0.1 of the network module to TEST and then PROD.
10. In the Portal, confirm that network module is now pinned at v1.0.1 across DEV, TEST, and PROD and now include the tag: manager = John Robinson (Network v1.0.1)
11. Confirm the security module remains unchanged and pinned at v1.0.0 with the tag: manager = Michael Coulling-Green (SecMod v1.0.0)

Takeaways

- Modules can evolve safely without changing consumer contracts.
- Promotion is explicit and intentional.
- Independent repositories enable independent lifecycles.
- Terraform plan output is a reliable audit mechanism.
- State isolation enables safe multi-environment promotion.

Congratulations, you have completed this lab

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026